

Teaching a machine to imagine Mario

AQ-Mario — a world model for Super Mario Bros 1-1, and the instruments that tell you when it is lying

Experiment report · 8 September 2026 · all numbers measured, none estimated

The goal is a model that has learned what *happens* in Mario — not one that can draw Mario. The test is whether Mario's height off the ground can be read back out of the model's internal state. A previous attempt at this scored **0.188** out of 1. This one scores **0.943**.

HEIGHT READABLE FROM
THE MODEL

0.943

prior attempt: 0.188 · 5.0×
better

FUTURE IT CAN PREDICT

0.719

prior attempt: 0.455

TRAINING USED SO FAR

4%

half of one pass through the
data

1. What is actually being built

A **world model** is a machine that has learned the rules of a world well enough to run it forward in its head. Show it three frames of Mario and tell it which buttons get pressed next, and it should be able to say what happens — without touching the game.

The unusual part is *where* it predicts. It does not draw the next picture. It compresses each frame into **192 numbers** and predicts those. Drawing pixels wastes most of the model's effort on clouds and brick textures; the 192 numbers are supposed to hold only what matters — where Mario is, how fast, what is about to kill him.

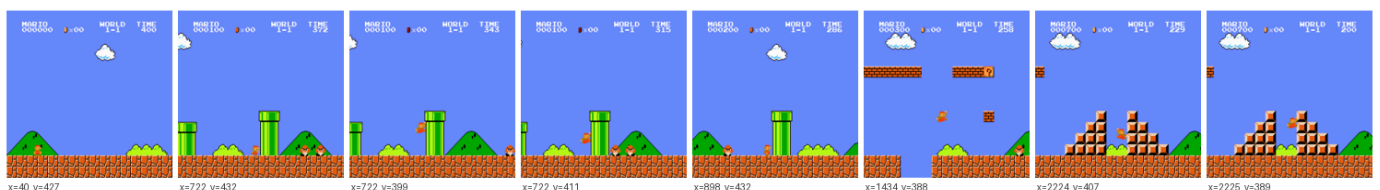


Figure 1. One real training episode. Each frame is stored with the true values pulled straight out of the console's memory: **x** is distance along the level, **y** is height on screen. Those true values are never shown to the model — they are the answer key used to grade it afterwards.

THE ONE IDEA TO TAKE AWAY

You cannot grade a world model by asking how confident it is. You grade it by opening it up and checking whether the facts you know are *findable inside it*. That is what "0.943" means: given only the model's 192 numbers, Mario's height can be recovered with 94.3% accuracy. The model was never told about height. It worked it out.

2. Why the previous attempt failed

The earlier effort, LeMario, built a similar model and then tried to plan with it — to search for a button sequence that would get Mario further. Planning failed, repeatedly. Only afterwards, by hand, did anyone check whether the model knew how high Mario was. It scored 0.188, which is close to knowing nothing.

The model had learned to track Mario *horizontally* almost perfectly and had thrown vertical position away. In a game that is mostly jumping, that is fatal — and it was invisible until months of planning work had been spent.

THE CHANGE IN METHOD

This project inverts the order. The check that took LeMario months of failed planning to reach runs **automatically, on every saved model, from epoch two**. If height is not recoverable, the run is stopped. Nothing downstream gets built on a broken foundation. That is the entire thesis, and everything below is in service of it.

3. How the data was made

Two robots played Mario 8,000 times on a rented fleet of 16 machines.

PLAYER	EPISODES	WHAT IT CONTRIBUTES	REACHES
Random — biased toward running right and jumping	4,000	Falls in every hole, hits every enemy. Teaches the model how death looks.	~45%
Trained — a reinforcement-learning agent	4,000	Actually gets somewhere. Covers the back half of the level.	100%

The second player was necessary and not obvious: the random one never gets past the first pipe complex, so a model trained only on its games would be graded on the opening 45% of the level and the score would not be comparable to anyone else's. The trained agent was deliberately **stopped early**, the moment it could reliably reach three-quarters of the way — an agent that finishes perfectly every time supplies 4,000 copies of one identical run, which teaches the model almost nothing.

Result: 2.7 million frames from the random player alone, 14.7 GB. Raw, that is 992 GB of pictures; the compression works because NES artwork is extremely repetitive.

4. The instruments

Four numbers are watched during training. Two of them are easy to misread, and misreading them is what nearly sank this experiment.

INSTRUMENT	PLAIN ENGLISH	GOOD
prediction error	How wrong the model's guess about the future is.	lower
explained future	Share of what happens next that the model correctly anticipates. 0 = guessing the average, 1 = perfect.	higher
spread	How many of the 192 numbers are actually carrying information.	higher
shape	Whether the 192 numbers are distributed healthily rather than piling up in a corner.	near 1.0

THE TRAP

A model that gives up entirely scores a perfect prediction error. If it maps every possible frame of Mario to the same 192 numbers, then predicting the future is trivial — the future is always the same thing. Prediction error collapses toward zero and "explained future" climbs toward 1.

This is not hypothetical. It happened here on the first real run: the model scored **0.939** on explained future — which would have beaten the previous attempt's 0.455 outright — while **spread** sat at 1.17 out of a healthy 27. Every frame of Mario had been crushed onto a single line. The model knew nothing and

the headline number looked excellent. **Spread is the honesty check; without it every other number can be faked.**

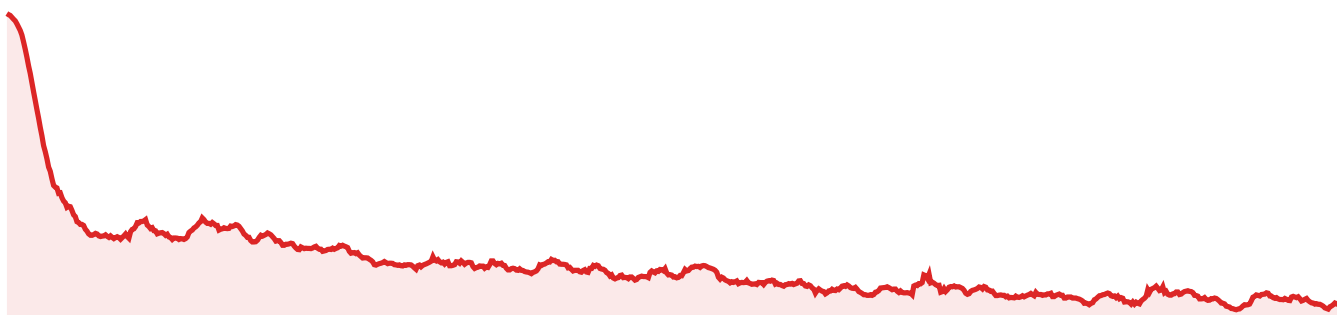
5. What went wrong, and the fix

Finding the collapse took seven steps. Each one is a thing that was quietly wrong.

1. **The safety check had a hole.** It asked only "is the error falling?" A collapsing model passes that perfectly. It now also demands that spread does not shrink.
2. **Two instruments were reading the wrong sample size** and reported a *healthy* model as $6\times$ worse than it was — a fake alarm sitting next to a real one.
3. **The anti-collapse safeguard was too weak to see the problem.** Its sensitivity depends almost entirely on how many examples it sees at once. At the batch size in use it could barely distinguish a fully collapsed model from a healthy one.
4. **Turning that safeguard up did not help**, and the arithmetic says why: collapsing earns the model 0.80 of prediction error, while every safeguard combined cost it 0.02. It was outgunned 45 to 1.
5. **The real cause was structural.** The model was being graded against a target it was also allowed to move. Making the target constant is the easiest way to hit it. Freezing the target removed the incentive entirely.

PREDICTION ERROR — falling

1.31 → 0.27



SPREAD — rising (the honesty check)

11 → 26



SHAPE — settling toward 1.0



Figure 2. The healthy pattern, from the actual run. Error falls *while* spread rises. The failure looked identical on the top chart and went to 1 on the middle one. The flattening at the right is deliberate — the learning rate is wound down to zero to end the run cleanly, not the model running out of ideas.

6. Results

The model is then frozen and interrogated: given only its 192 numbers, can a simple reader recover facts about the game? R^2 of 1.0 means perfect recovery, 0.0 means none.

QUESTION PUT TO THE MODEL	THIS MODEL	LEMARIO	TARGET	
How high is Mario?	0.943	0.188	0.90	PASS
How far along the level is he?	0.963	0.997	0.97	marginal
Where is the camera?	0.962	—	0.90	PASS
Is he about to die?	0.967	—	—	31× baseline

THE HEADLINE

The single question LeMario got wrong — **how high is Mario** — went from **0.188 to 0.943**. And this is from a model trained on roughly **4% of the planned schedule**: half of one pass through half of the data. It was not a hard problem once it was being measured.

The one thing that does not pass

A fourth check asks whether the model can tell apart two moments that are far apart in the level. It fails. The diagnosis is specific and useful: the model's sense of distance grows reliably out to about **350 pixels** of separation and then goes flat — two places roughly two screen-widths apart start to look alike. The console screen is 256 pixels wide, so this is the camera.

This matters for what comes next. The planner is designed to steer toward goals about **200 pixels** ahead at a time — a number picked on intuition before any of this was measured. The measurement says the model is trustworthy out to 350. The guess and the measurement agree, which is the useful outcome: the plan is safe, and now it is safe *for a stated reason*.

7. Where this is

STAGE	WHAT IT DELIVERS	DONE
1 · Ground truth	8,000 games with trustworthy answer keys	~90%
2 · World model	The model itself, trained without collapsing	~75%
3 · Instruments	Automatic checks that stop a bad run early	~55%
4 · Control	Use the model to actually play; steer it by hand	~15%

Everything is written and tested — 141 automated checks, all passing. What remains is mostly compute: the full training run, and then using the model to play.

What would make it better

- **Finish training.** These results are from 4% of the planned run. Every trend was still improving when it stopped.
- **Use the second dataset.** The trained player's 4,000 games — covering the back half of the level — are being collected now and were not in this model at all.
- **Faster hardware.** ~3 hours instead of ~12 for the full run.
- **A sampling fix already made.** Deaths are rare — 1 in 130 frames — so half of all training batches contained no death at all and the model got no signal about dying on those steps. Now corrected; it was also the cause of the jagged training curve.

HONEST SUMMARY

The thing this project set out to prove — that you can catch a world model's blind spot by inspecting it rather than by watching it fail downstream — **worked**. The blind spot that cost LeMario months was caught automatically, and then fixed, in a run that used 4% of its training budget. The instruments themselves needed four separate corrections along the way, and each of those corrections is written down, because an instrument you have not calibrated is not evidence.